

“SimSnap” Framework: Designing Interaction Methods for Cross-device Applications

May Yu
Synaesthetic Media Lab, Toronto
Metropolitan University
may.yu@torontomu.ca

Afroza Sultana
Synaesthetic Media Lab, Toronto
Metropolitan University
afroza.sultana@torontomu.ca

Stacy Cernova
Synaesthetic Media Lab, Toronto
Metropolitan University
stacy.cernova@torontomu.ca

Megan Wang
Synaesthetic Media Lab, Toronto
Metropolitan University
megan1.wang@torontomu.ca

Alex Bakogorge
Synaesthetic Media Lab, Toronto
Metropolitan University
abakogorge@torontomu.ca

Tudor Tibu
Synaesthetic Media Lab, Toronto
Metropolitan University
tudor.tibu@torontomu.ca

Aneesh P. Tarun
Synaesthetic Media Lab, Toronto
Metropolitan University
aneesh@torontomu.ca

Ali Mazalek
Synaesthetic Media Lab, Toronto
Metropolitan University
mazalek@torontomu.ca

ABSTRACT

Despite significant developments in cross-device interaction techniques, disengagement interactions- that disconnect devices from the application- remain underexplored. This paper introduces “SimSnap”, a touch-based cross-device interaction framework for connecting and disconnecting devices. We extended existing connection methods and developed a novel approach to disconnection. We identified design considerations for touch-based disconnecting and present our recommendations to address these issues.

CCS CONCEPTS

• **Human-centered computing** → Collaborative and social computing; Collaborative and social computing theory, concepts and paradigms; Computer supported cooperative work.

KEYWORDS

Cross-device interaction, Cross-device framework, Connection methods, Disconnection methods, Spatial device rearrangement

ACM Reference Format:

May Yu, Afroza Sultana, Stacy Cernova, Megan Wang, Alex Bakogorge, Tudor Tibu, Aneesh P. Tarun, and Ali Mazalek. 2024. “SimSnap” Framework: Designing Interaction Methods for Cross-device Applications. In *The 37th Annual ACM Symposium on User Interface Software and Technology (UIST Adjunct '24)*, October 13–16, 2024, Pittsburgh, PA, USA. ACM, New York, NY, USA, 3 pages. <https://doi.org/10.1145/3672539.3686319>

1 INTRODUCTION AND MOTIVATION

The ubiquitous nature of mobile devices has led researchers in HCI to explore new applications of devices using cross-device interactions, where users can work with various mobile devices to achieve a common goal [9]. Frederick Brudy identified three phases of cross-device interaction techniques: (1) configuration— connecting individual devices to a collaborative framework, (2) content engagement— interacting with the multi-device framework through various gestures, and (3) disengagement— disconnecting a device from the cross-device framework [1]. The majority of prior research on cross-device platforms has focused on the first two phases: designing and implementing various types of device connection methods and cross-device interactions. However, little attention has been given to designing new disengagement methods [1].

The limited prior work that developed disconnecting methods mainly designed pickup or shaking gestures using the device’s accelerometer/gyroscopic data [4, 6, 8]. This disconnection method restricts possibilities for new interactions, given that the devices must remain stationary to stay connected. Other applications which applied different frameworks for connections [3] and content engagement [5, 10, 11], provided little information about their implementation of disconnecting as an interaction technique [1].

This paper presents “SimSnap”, a complete framework for developing cross-device applications that presents a connection method which expands upon prior work [7] and a novel disconnection method. We present the design considerations and solutions of our framework.

2 SIMSNAP FRAMEWORK

“SimSnap” is a cross-device interaction framework which employs a flexible touch-based event system over a network to facilitate collaboration across multiple devices. Our goal for “SimSnap” was to design connecting and disconnecting methods which allow more interactions such as dynamic spatial re-configuration [7] and shake/tilt-based interactions [2]. We designed a connection method called ‘snapping’ which uses a similar approach to Pinch’s

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

UIST Adjunct '24, October 13–16, 2024, Pittsburgh, PA, USA

© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0718-6/24/10

<https://doi.org/10.1145/3672539.3686319>

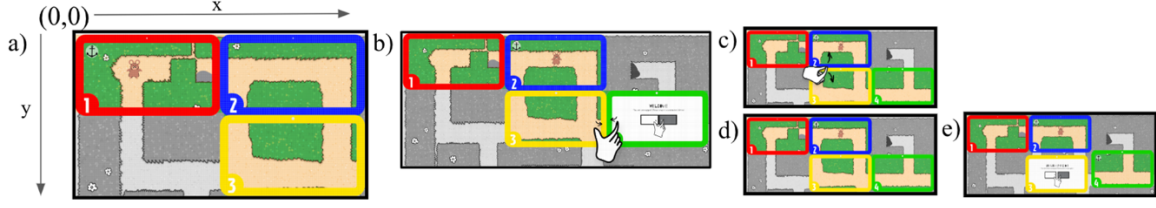


Figure 1: A maze application designed to showcase “SimSnap’s” interaction techniques. a) 3 devices are connected to the canvas, Device 1’s top left corner is placed at (0,0) on the canvas. b) To connect Device 4 to the canvas, a snap is performed between Device 3 and Device 4. c) Device 4 is now connected to the canvas and snapped to the right of Device 3. d) Device 3 wishes to disconnect from the canvas, an unsnap is performed between device 3 and device 2. Since Device 2 is an anchor, Device 3 is removed. e) Device 3 is now unsnapped, and Device 4 is automatically orphaned and set as an anchor.

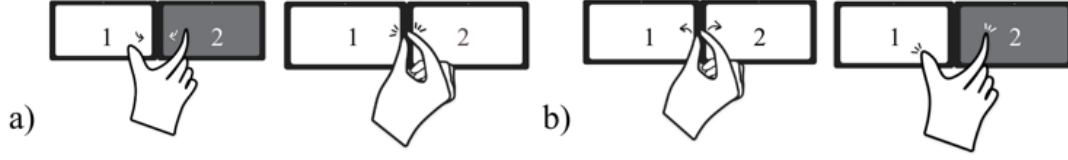


Figure 2: a) The snapping gesture being performed on two tablets. Device 2 is connected to the canvas and snapped to Device 1. b) The unsnapping gesture being performed on two tablets. Device 2 is unsnapped from Device 1.

touch-based connecting [7], and a novel touch-based disconnection method called ‘unsnap’. The “SimSnap” framework is an interactive multi-device display system which utilizes coordinates on a canvas to determine device orientation and connectivity. “SimSnap” uses a canvas which stretches across multiple screens; in which connected devices are assigned a coordinate on the canvas and shown a portion of an image on the canvas. Subsequent devices have their positions on the canvas calculated based on size and relation to other devices (Figure 1).

2.1 Connection

“SimSnap” has 3 levels of connections within the framework.

Connecting to a SimSnap Application: Upon first accessing an application built on the “SimSnap” framework, all devices are connected to a server, referred to as ‘being online’. The first device which connects to the server views a portion of the canvas, with its top left corner positioned at (0,0) on the canvas (Figure 1(a)). All subsequent devices that connect to the SimSnap application are shown a welcome screen (see Device 4 in Figure 1(b)). Once online, devices can perform touch gestures to connect to or disconnect from the canvas.

Connecting to the Canvas: A device connected to the canvas views a portion of the canvas and can interact with objects present on the canvas (see mouse object in Figure 1). To connect a new device to the canvas, a snapping gesture is performed between the new device and an already connected device (Figure 2(a)). The position of the new device on the canvas is calculated based on the other device’s canvas position and both of the devices’ sizes and orientations.

Snapping: Devices directly adjacent to each other on the canvas are called “snapped” (Figure 2(a)) and can perform cross-device interactions, such as moving objects from one device to another.

2.2 Disconnection

Unsnapping: “Snapped” devices can perform an ‘unsnap’ gesture to disconnect one of the devices from the canvas. An unsnap is performed on two snapped devices through adjacent swiping motions away from the bezel on the side of their connections (Figure 2 (b)). An unsnapped device no longer views portions of the canvas, instead showing a screen indicating their “unsnapped” state to the user (see Device 3 in Figure 1(e)). If a device is snapped to multiple devices, unsnapping at any of the connected edges removes all of the devices remaining snap connections to other devices (see Figure 1(d) and (e)).

When an unsnapped gesture is performed between two devices on the canvas (see Device 2 and Device 3 in Figure 1(b)) both devices send the server the same information. Meaning, the server has no indication of which device wishes to be disconnected, preventing unsnapping from functioning correctly.

2.3 Anchoring Devices

We introduce ‘anchoring’, a way to assign specific devices a priority to the canvas, as a solution to being unable to determine which device we wished to disconnect from the canvas from the unsnapping gesture alone. During an unsnap, devices assigned the ‘anchor’ role stayed connected to the canvas while the other device either removed itself from or changed its position on the canvas.

The method for assigning anchors allows future applications to create their own parameters for assigning anchors to specific devices. To demonstrate this flexibility, we designed a simple maze application. Any device containing the draggable mouse object was automatically set as an anchor and could not be removed from the canvas. (see Device 1 in Figure 1(a) and Device 2 in Figure 1(b), (c), (d), (e)). Other devices which wished to become an anchor could be set with a toggle button.

A set of rules for anchoring was established to ensure applications could function correctly and handle all use cases.

2.3.1 Anchoring Rules. Rule 1: Anchors cannot be set on tablets directly next to each other on the canvas, to prevent a situation where two devices with the same canvas priority try to disconnect.

Rule 2: A device connected to the canvas but no longer snapped to any device, called ‘orphaned’, automatically becomes an anchor (see Device 4 in Figure 1(e)).

Rule 3: When an unsnap is performed and neither device is an anchor, the device that is snapped to more devices remains stationary on the canvas (if Device 4 were to disconnect from Device 3 in Figure 1(c)).

3 CONCLUSION

This paper introduces “SimSnap”, a new framework with design considerations addressing challenges with connection and disconnection methods in a cross-device platform. Current applications using this framework include the tabletop Snap-to-Eat maze game showcasing the flexibility of the novel anchoring system in facilitating disconnections. Future developments of the framework will expand upon the existing connection and disconnection methods. Moreover, we will design novel cross-device interactions through collecting embedded sensor data, such as accelerometer/gyroscopic data for a wide range of applications including collaborative games and educational software.

REFERENCES

- [1] Frederik Brudy, Christian Holz, Roman Rädle, Chi Jui Wu, Steven Houben, Clemens Nylandsted Klokmoose, and Nicolai Marquardt. 2019. Cross-device taxonomy: Survey, opportunities and challenges of interactions spanning across multiple devices. In *Conference on Human Factors in Computing Systems - Proceedings*. Sam Anzaroot and Andrew McCallum. 2013. UMass Citation Field Extraction Dataset. Retrieved May 27, 2019 from <http://www.iesl.cs.umass.edu/data/data-umasscitationfield>
- [2] Linda Di Geronimo and Moira C. Norrie. 2016. Rapid development of web applications that use tilting interactions in single and multi-device scenarios. In *Proceedings of the Workshop on Advanced Visual Interfaces AVI*. <https://doi.org/10.1145/2909132.2926088>
- [3] Ken Hinckley, Gonzalo Ramos, Francois Guimbretiere, Patrick Baudisch, and Marc Smith. 2004. Stitching: Pen gestures that span multiple displays. In *Proceedings of the Workshop on Advanced Visual Interfaces AVI*. <https://doi.org/10.1145/989863.989866>
- [4] Heidi Pi Jensen, Marius Pallisgaard Olsen, and Mikael B Skov. 2016. PinchPan: Investigating Children’s Collaboration in Cross-Device Interaction. In *Proceedings of the 13th International Conference on Advances in Computer Entertainment Technology*, 1–6. Retrieved May 7, 2023 from <https://doi.org/10.1145/3001773.3001789>
- [5] Seungki Kim, Donghyeon Ko, and Woohun Lee. 2017. Utilizing smartphones as a multi-device single display groupware to design collaborative games. In *DIS 2017 - Proceedings of the 2017 ACM Conference on Designing Interactive Systems*. <https://doi.org/10.1145/3064663.3064716>
- [6] Heidi Selmer Nielsen, Marius Pallisgaard Olsen, Mikael B Skov, and Jesper Kjeldskov. 2014. JuxtaPinch: exploring multi-device interaction in collocated photo sharing. *Proceedings of the 16th international conference on Human-computer interaction with mobile devices & services*: 183–192. Retrieved May 7, 2023 from <https://doi.org/10.1145/2628363.2628369>
- [7] Takashi Ohta and Jun Tanaka. 2012. Pinch: An interface that relates applications on multiple touch-screen by ‘pinching’ gesture. In *International Conference on Advances in Computer Entertainment Technology*, 320–335. Retrieved May 7, 2023 from https://doi.org/10.1007/978-3-642-34292-9_23
- [8] Takashi Ohta and Jun Tanakay. 2015. MovieTile: Interactively adjustable free shape multi-display of mobile devices. In *SIGGRAPH Asia 2015 Mobile Graphics and Interactive Applications*, SA 2015. <https://doi.org/10.1145/2818427.2818436>
- [9] Florian Scharf, Christian Wolters, Michael Herczeg, and Jörg Cassens. 2013. Cross-Device Interaction: Definition, Taxonomy and Application. *AMBIENT 2013: The Third International Conference on Ambient Computing, Applications, Services and Technologies*, c.
- [10] Mikael B. Skov, Jesper Kjeldskov, Jeni Paay, Heidi P. Jensen, and Marius P. Olsen. 2015. Investigating cross-device interaction techniques: A case of card playing on handhelds and tablets. In *OzCHI 2015: Being Human - Conference Proceedings*. <https://doi.org/10.1145/2838739.2838763>
- [11] Paul Strohmeier. 2015. DisplayPointers: Seamless cross-device interactions. In *ACM International Conference Proceeding Series*. <https://doi.org/10.1145/2832932.2832958>